Automata Theory

Introduction

CS32022

By Thisara Watawana

01 July 2026

0. Theory of Computation

“What are the fundamental capabilities and limitations of computers?”



0. Theory of Computation

- This fundamental question goes back to the 1930s, decades before modern electronic computers were built.
- It began when researchers first attempted to mathematically define the exact meaning of computation.
- Rapid technological advances have shifted this investigation from pure mathematical theory into a matter of urgent practical concern.



0. Theory of Computation

The Evolutionary Timeline of Computing

- The Abstract Blueprint (1936) - Alan Turing & Alonzo Church.
- The Stored-Program Architecture (1945) - John von Neumann.
- The Silicon & Mass Computing Microprocessor (1971) - Federico Faggin & Ted Hoff.
- Vector Processing & Supercomputing (1976) - Seymour Cray.
- The Quantum Shift (1980s–Present) - Richard Feynman & David Deutsch

0. Theory of Computation

The Evolutionary Timeline of Computing

Invented the "Turing Machine" and the λ-calculus.

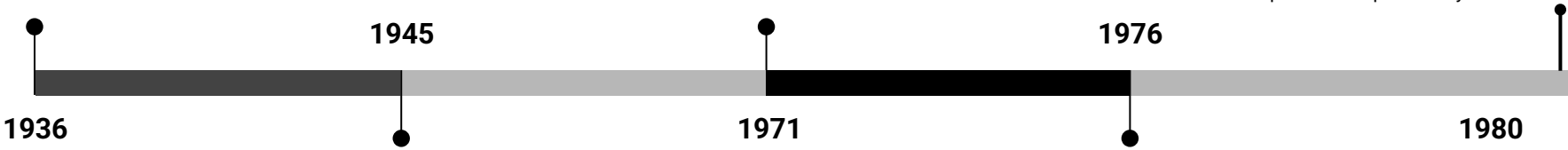
They defined what an "algorithm" is mathematically before a single digital computer existed

Engineered the Intel 4004, the world's first commercial microprocessor.

Stripped a room-sized computer's CPU onto a single silicon chip smaller than a fingernail, shifting computers from massive enterprise tools to accessible personal devices.

Proposed computing based on quantum mechanics rather than classical physics.

Replaced classical binary bits (0 or 1) with quantum bits (qubits) that exist in superposition, threatening to break modern cryptography by solving otherwise "intractable" problems exponentially faster.



Formulated the "Von Neumann Architecture."

He introduced the concept of sharing a single memory space for both data and program instructions the foundational structural blueprint for almost all modern CPUs today.

Designed the Cray-1 Supercomputer.

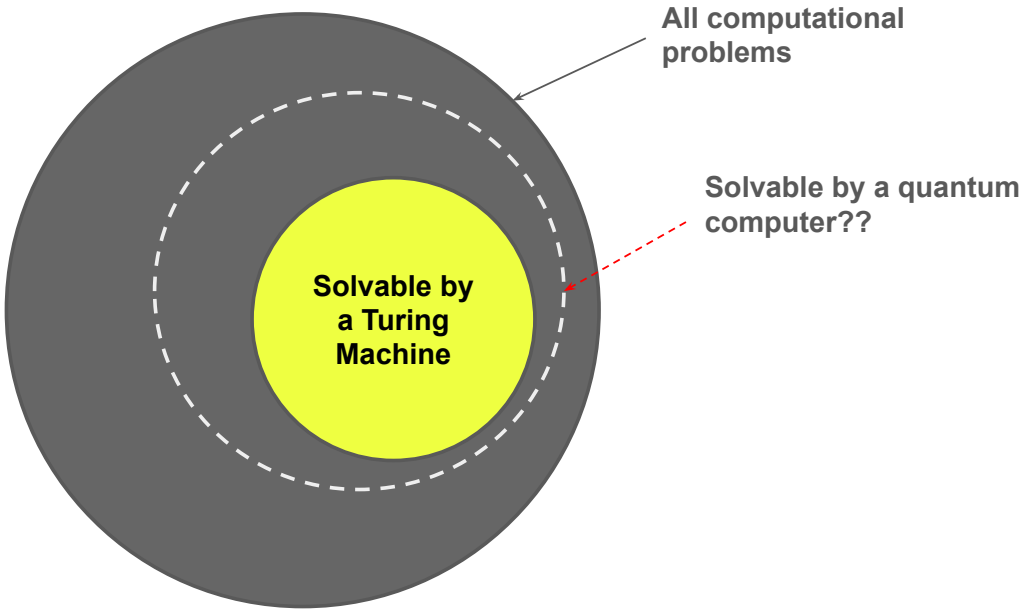
It used a unique vector processing design to execute hundreds of operations simultaneously in parallel, paving the way for modern high-performance computational modeling.

0. Theory of Computation

Exercise 01

A quantum computer can solve a factoring problem in minutes that would take a classical supercomputer billions of years to compute. Based on this, do you think a quantum computer can solve a problem that a Turing Machine is mathematically incapable of solving? Take 60 seconds to talk to the person next to you.

0. Theory of Computation

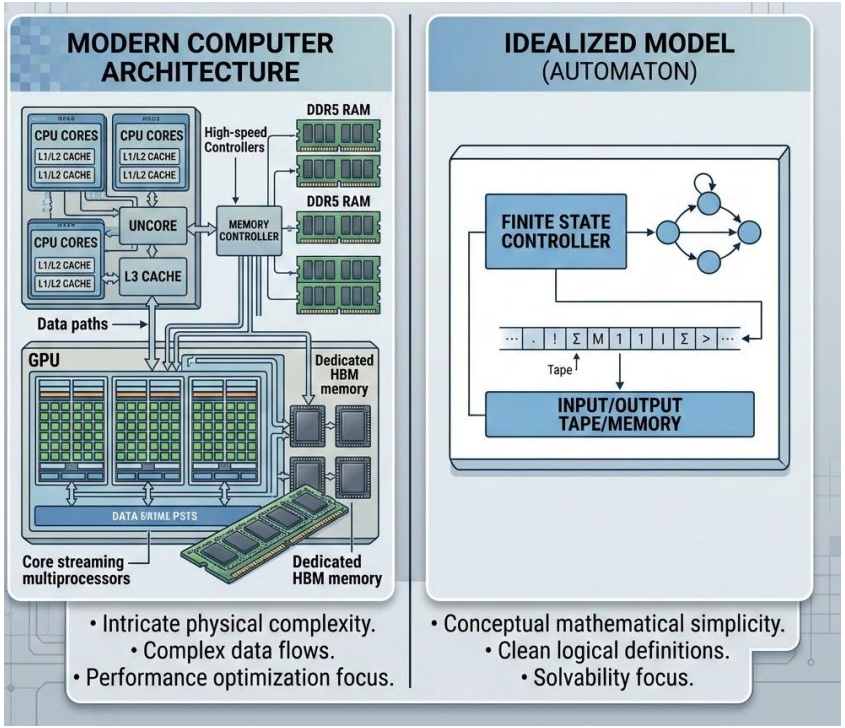


- Quantum mechanics changes the speed of computation (Complexity).
- But it cannot alter the absolute laws of what is logically possible to compute (Computability).
- A problem that is mathematically impossible for a 1930s Turing machine is equally impossible for a 2026 quantum supercomputer.

0. Theory of Computation

What is a "Computer"?

- Real-world modern computers are far too intricate to analyse directly with a clean mathematical theory.
- We strip away the physical hardware, operating systems, and lines of code to focus purely on logic.
- We study simplified, mathematical blueprints of computers called computational models.



0. Theory of Computation

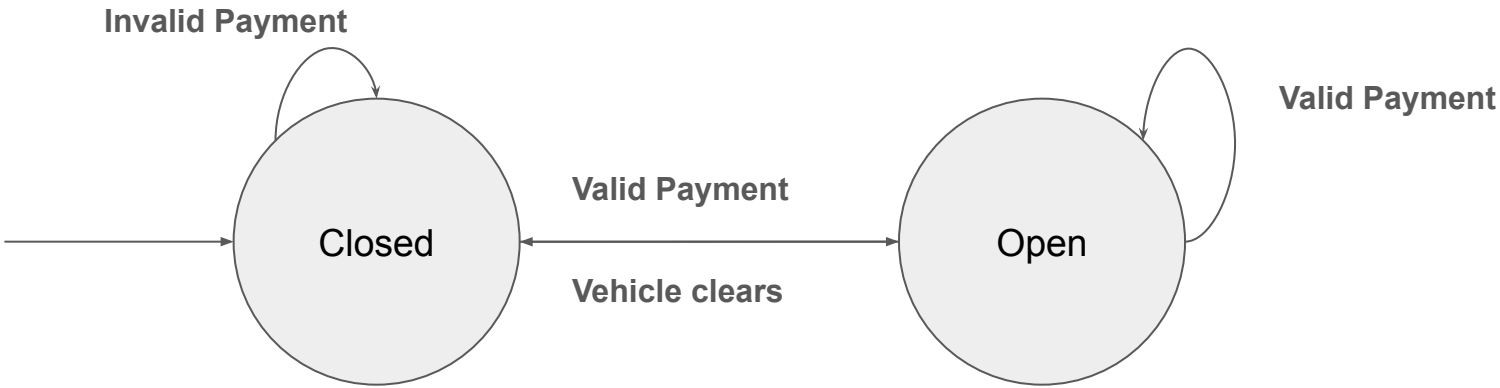
Exercise 02

Imagine you are programming the controller for an automated highway toll gate. The gate starts out with the barrier arm down to prevent cars from passing through without paying.

When a driver taps their payment card, the barrier arm lifts. Once the vehicle clears the safety sensor on the other side, the arm immediately lowers again to block the next car.

0. Theory of Computation

Exercise 02



0. Theory of Computation

The Three Pillars of Computation

- The theory of computation is traditionally divided into three central areas: Complexity, Computability, and Automata.
- All three areas are deeply linked by a single, foundational question: What are the fundamental capabilities and limitations of computers?

0. Theory of Computation

1. **Complexity Theory**

- Classifying computational problems based on how easy or hard they are to solve.
- Sorting a million numbers is computationally easy. Creating a university class schedule with thousands of conflicting room/time constraints is computationally hard and can take centuries to solve.
- While we usually want problems to be easy, fields like cryptography specifically require problems to be computationally hard so that codes cannot be cracked without a secret key.

0. Theory of Computation

Exercise 03

- **Task A**: Take a messy list of student names and alphabetize it from A to Z.
- **Task B**: Take a list of 500 delivery packages and find the absolute shortest driving route to deliver all of them and return to the depot.

Discuss the complexity of solving the above tasks.

0. Theory of Computation

2. Computability Theory

- Classifying problems by those that are solvable by an algorithm versus those that are completely unsolvable.
- In the early 20th century, logicians like Gödel, Church, and Turing discovered that certain basic mathematical problems can never be solved by a computer.
- There is no algorithm capable of determining whether any given mathematical statement is true or false.

0. Theory of Computation

Exercise 04

- **Task A**: Given an encoding of a directed graph, a starting vertex ***s***, and a target vertex ***t***, determine whether a valid directed path exists from ***s*** to ***t***.
- **Task B**: Write an IDE or optimization tool that scans a massive enterprise codebase and identifies every single function, variable, or line of code that is completely unreachable ("dead code") under every possible runtime condition, allowing it to be safely deleted.

Discuss the computability of the above problems.

0. Theory of Computation

Exercise 04

- **Task C**: Build an antivirus scanner that perfectly analyzes any submitted executable file and determines if it will ever execute a malicious action (like encrypting a hard drive or modifying system files).
- **Task D**: Given a binary integer, determine whether it is prime or composite.

Discuss the computability of the above problems.

0. Theory of Computation

3. Automata Theory

- Defining the exact properties of the mathematical models of computation.
- To evaluate how much time a computer needs (Complexity) or whether a problem is solvable at all (Computability), we must first have a strict, error-free mathematical definition of what a computer actually is.

0. Theory of Computation

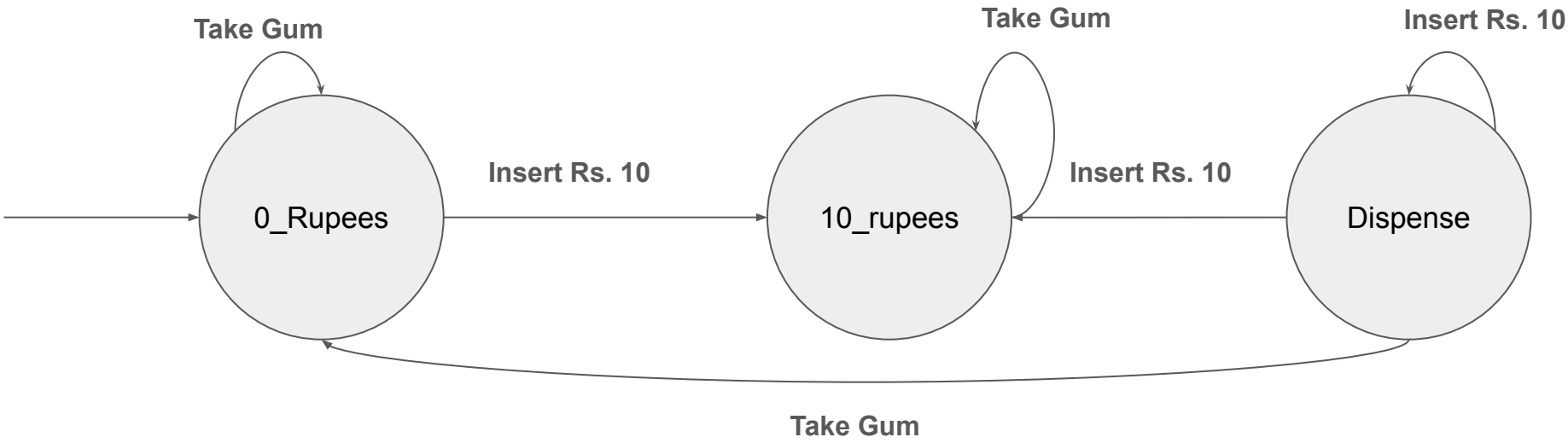
Exercise 05

You are designing the electronic control board for a simple snack vending machine. A pack of chewing gum costs exactly Rs. 20. The machine only accepts Rs. 10 coins and has no coin-return mechanism. It will ignore any extra coins once the cost is met. Once the candy is dispensed, the machine resets.

Identify the system's operational states, the valid user inputs, draw the transition map, and compile the state table.

0. Theory of Computation

Exercise 05



0. Theory of Computation

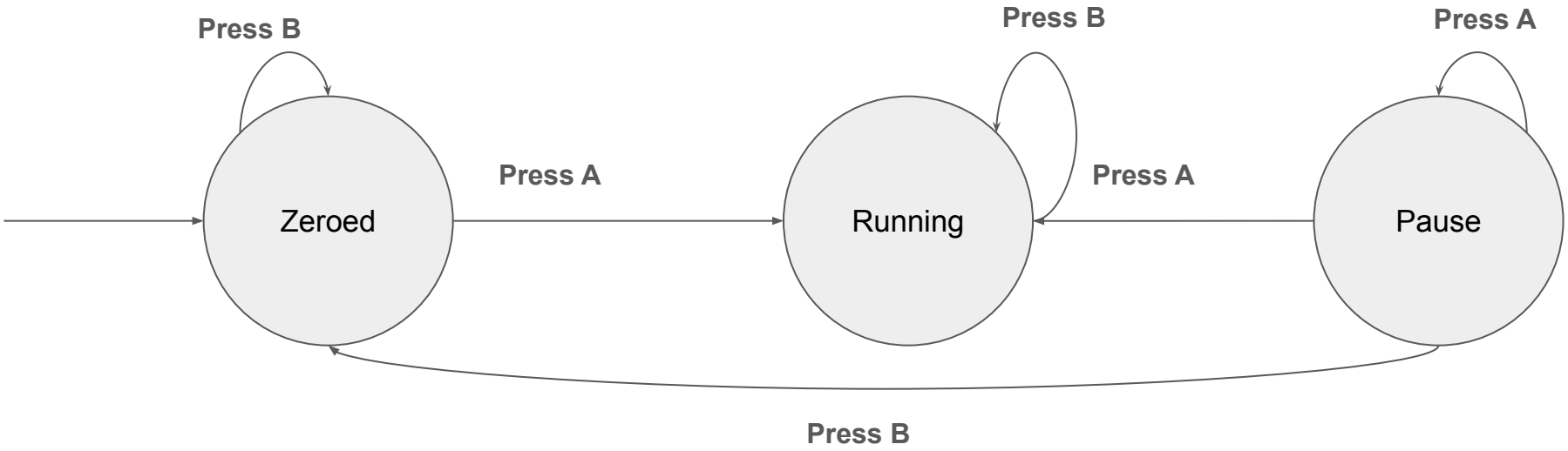
Exercise 06

You are building the user interface logic for a digital sports watch. The stopwatch has exactly two physical buttons: Button A (Start/Pause) and Button B (Reset). The watch starts completely cleared at zero. Pressing Reset should have no effect while the timer is actively ticking.

Map out the background state behavior of this device when a user interacts with these physical buttons.

0. Theory of Computation

Exercise 06



0. Theory of Computation

Exercise 07

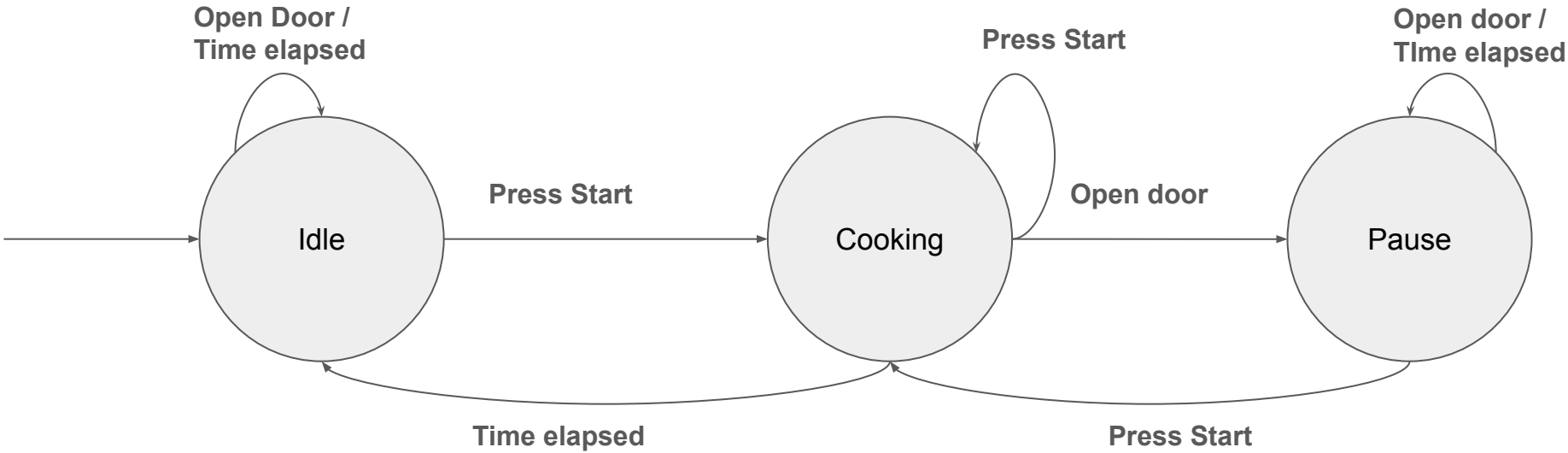
You are programming the safety control computer for a kitchen microwave oven. The appliance starts sitting idle. To heat food, a user must explicitly Press Start (which activates the heating magnetron tube).

For safety, if a consumer physically opens the door while the system is cooking, the magnetron must instantly shut off and freeze the remaining time. Closing the door does not automatically restart cooking; the user must manually trigger Start again.

Isolate the security states, environmental events, and the ultimate logic table for this appliance.

0. Theory of Computation

Exercise 07



Questions?

thisaraw@boffin.lk